
CSC 412 – Operating Systems

Programming Assignment 01, Fall 2025

Monday, September 8th, 2025

Due date: Tuesday, September 16th, 11:59pm.

1 What this Assignment is About

1.1 Objectives

The objectives of this assignment are for you to

- Get started with C programming, in particular array creation and manipulation, and file I/O;
- Make sure that you properly follow assignment specifications;
- Get feedback before Prog 02 is due.

This is an individual assignment.

1.2 Handout

The handout for this assignment is this document as a pdf file.

1.3 My rules

Whether we do the grading on Gradescope or manually (undecided at the time of this writing), when I give specifications, you are expected to follow these direction. This includes:

- folder organization,
- names of source, header, and script files,
- list of arguments (number, order, and type),
- format of your output,
- file type of the report (pdf, *not* doc, docx, rtf, or txt).

1.4 In case of doubt or ambiguity

If you have any doubt, or if you think that there is something ambiguous, or even wrong, about my specifications, post a question on the Brightspace forum topic dedicated to this assignment.

1.5 Organization for this assignment

The root folder for this assignment is named `Prog01`. This folder will contain

- A folder named `Programs`, where the different versions that you implement will go;
- A folder named `Scripts`, where the `build.sh` script will go;
- Your report, a file in the Acrobat file format named `report.pdf`

You will submit on BrightSpace a zip archive (nor `.rar`, `.Z`, `.tar` or other archiving format) of the project folder, and nothing besides what's required (in particular, absolutely no binary executable, output files, etc.).

1.6 On Unix vs. Linux

Throughout this assignment (and in the future), whenever I say “On Unix,…” please be aware that strictly and legally speaking, Linux is *not* Unix™, but merely “Unix-like,” because most common, general-public distributions have not been certified by the [Open Group](#) (unlike, say, Apple's macOS, formerly known as MacOS X). This being said, over 99% of Unix terminal commands are found on both system, with only minor variations in the list of options.

2 Part I: C Program

2.1 About the Task

In this assignment, we will perform a very simple geometric task. Our input consists of lists of 2D points defined by their coordinates (Figure 1(a)) and we will compute a bounding box for these points either by computing ranges of x and y coordinates (Figure 1(b)) or the center, width, and height of the box (Figure 1(c)).

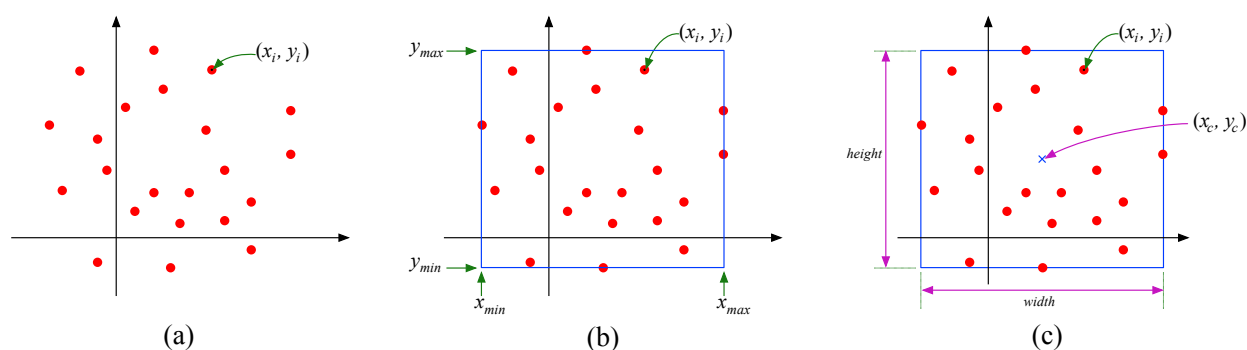


Figure 1: The task. (a) 2D points identified by their Cartesian coordinates; (b) The points' bounding box defined by coordinate ranges; (c) The points' bounding box defined by the coordinates of its center, and its width and height.

2.2 Data file format

The main purpose of this course is for you to practice system programming, not to spend a lot of time dealing with intricacies of file I/O and data validation. Therefore, when I create data file formats for input or output, I always try to make them human-readable (text files rather than binaries) and easy to read (e.g., give the number of lines of data rather than having to test for end of file). In this spirit, the data files we will encounter in this assignment all have the same simple format:

- All data files are text files¹,
- The first line of the file gives the number n of 2D points,
- The following n lines of the file give the x and y coordinates of the points as floating point values, separated by a blank character.

For example, the following could be the contents of a valid input data file defining five 2D points:

```
5
3.4 -12.5
-7.2 4.5
3 8.3
0.0 4.5
25.3 3.2
```

2.3 Version 1: Single data file

2.3.1 Name of the source file

To simplify the graders's job I impose that your source file should be named `prog01v1.c`. As mentioned earlier, this file should reside in the `Programs` subfolder of your project folder.

2.3.2 Format of the output

In this version of the program, you are expected to compute the range of coordinates of the points' bounding box. Your output will consists of a single text data file containing a single line listing the x_{min} , x_{max} , y_{min} , y_{max} values, in that order, separated by a blank character. For example, in the case of the input data file listed above, the output file should contain

```
-7.2 25.3 -12.5 8.3
```

Rounding issue: Point coordinates (and therefore, the bounding box's information) are represented as floating point numbers. This means that we have to deal with rounding problems (to give a simple example, even a number as innocent-looking as 0.1 cannot be stored exactly in a float or double format). In order to avoid problems of programs being penalized because of a difference on the 7th decimal position, for the entire assignment, we are simply going to request that you output a **single digit** of the fractional part, rounded down for lower bounds, and rounded up for upper bound. So, for example, if the value stored is 13.24, you should output 13.2 for an x_{min} or y_{min} value, and 13.3 for an x_{max} or y_{max} value. If the value stored is -14.58, you should output -14.6 for an x_{min} or y_{min} value, and -14.5 for an x_{max} or y_{max} value.

¹Furthermore, all these text files end with a terminating linefeed character.

2.3.3 Input to the program

Your program will take two arguments: the path to an input data file and the path to the expected output file. For example, assuming that your program has been compiled with executable name `prog01v1`, we would expect it to be launched with a command line such as

```
./prog01v1 dataFile1.txt Output/output1.txt
```

or

```
./prog01v1 DataFile/dataFile1.txt ../Output/output1.txt
```

or

```
./prog01v1 /user/NotYourAccount/Prog01Grading/dataFile1.txt output1.txt
```

In other words, your program must be able to handle any valid paths, whether relative or absolute, to an input data file and an output file.

Note: In this assignment, we will test your program with valid paths. For example, in the first example above, the input data file `dataFile1.txt` would exist (and contain valid data) and so would the `Output` folder where to write the output file. We will have an assignment dedicated to handling file system issues later this semester, but now is not the time for this kind of problems.

2.4 Version 2: Multiple data files

The only difference between this version and the previous one is that now the list of points is split over multiple data files. the bounding box that you must compute is that of the global list of points.

2.4.1 Name of the source file

Your source file should be named `prog01v2.c`.

2.4.2 Format of the output

The format of the output is the same as that of Version 1.

2.4.3 Input to the program

Your program will take in a list of arguments: first a list of $n \geq 1$ paths to input data files, and then the path to the expected output data file. For example, assuming that your program has been compiled with executable name `prog01v2`, we would expect it to be launched with a command line such as

```
./prog01v2 data1.txt data2.txt data3.txt Output/output123.txt
```

Note 1: Just as with Version 1, we will only test your program with valid paths and valid data files.

Note 2: The “I’ll just preallocate a large array that should be big enough for what’s to come” approach is completely unacceptable for this assignment and future assignments this semester². You are expected to find out the proper size of your arrays, then allocate them. And then resize when appropriate.

2.5 About compiler settings (and warnings in particular)

I will have ample opportunity to discuss this further in class, but I go a very different route for choosing a language version in C and C++. When programming in C, you will find that the overwhelming majority of code you will find online is “old style C,” often as old as C99. I choose to use C11, which represents a nice “middle ground” with the newer versions of the language (the next one is C17, the most recent stable version is C23) that introduce/shoehorn features coming from C++.

When programming in C++, I strive to use the latest stable version of the language, which at the time of this writing is C++23. This means that I have to keep working at staying current as new features are added to the language and some are deprecated.

If you end up developing professionally in C or C++, your employer will make it very clear what version of the language is targeted. For this semester, I am telling you that we are targeting C11 when programming in C (as in this assignment), and C++23 when programming in C++.

We will test your build and run scripts (discussed in the next section), but we will also build directly your programs. So, please note that:

- We build with compiler setting `-std=c11`
- We build with the argument `-Wall` to get all compiler warnings. A compiler warning is the compiler’s way of telling you “I am not quite sure what you want to do here, so I am going to take a wild guess.” The last thing you want is for the compiler to get a wild guess about your intentions regarding the way to handle, say, an unsigned to signed number conversion, or an implicit initialization.

There will be penalties for warnings reported by the compiler. Code that doesn’t compile will be scored 0 for execution and style.

2.6 Extra credit (5 points)

This EC builds on Version 1. The source file should be named `prog01_EC1.c`. The only change compared to V1 is that the bounding box information doesn’t come as coordinate bounds, but as the coordinates of the box’s center (one digit in fractional part, rounded to closest digit), and the box’s width and height (one digit in fractional part, rounded to closest larger digit). In the case of the list of points of Subsection 2.2, your output file should consist of a single line of text:

9.1 -2.1 32.5 20.8

²Except in a few very specific, limited cases that we will discuss in detail later this semester.

3 Part II: Scripts

3.1 Build script

This script, named `build.sh`, is meant to build executables for all the versions (including the ones for extra credit) that you completed. It is meant to be run from the project's root folder and build the executables there, with the following line of command:

```
Scripts/build.sh
```

3.2 Run scripts

3.2.1 Run script for Version 1

This script, named `script01_1.sh`, is meant to be run from the root of the project folder and takes two arguments: the path to an input data file and the path to the expected output file. For example:

```
Scripts/script01_1.sh dataFile1.txt Output/output1.txt
```

Your run script should call the build script, and then call the Version 1 executable with the proper arguments.

3.2.2 First run script for Version 2

This script, named `script01_2.sh`, is meant to be run from the root of the project folder and takes as arguments a list of paths to input data files and the path to the expected output file. For example:

```
Scripts/script01_2.sh dataFile1.txt dataFile2.txt dataFile3.txt\  
Output/output1.txt
```

Your run script should call the build script, and then call the Version 2 executable with the proper list of arguments.

Note: The backslash character in the above command's first line isn't a typo and it will probably create problems if you try to copy the command from this document and paste it in your terminal. The backslash character is used to break a long command into more than one line, and basically says: "the command continues on the next line." Make sure that the backslash immediately precedes the end of line (return) character (no blank character such as space or tab), otherwise the shell would misunderstand your command.

3.2.3 Second run script for Version 2

This script, named `script01_3.sh`, is meant to be run from the root of the project folder and takes two arguments:

- The path to a text file containing a list of paths to input data files;

- The path to the expected output file.

For example:

```
Scripts/script01_2.sh Data/pathList.txt Output/output1.txt
```

where the file `pathList.txt` could contain a list of paths to data files such as

```
dataFile1.txt  
dataFile2.txt  
dataFile3.txt
```

Your run script should call the build script, and then call the Version 2 executable with the proper list of arguments.

3.2.4 Extra credit (6 points)

This script, named `script01_EC1.sh`, is a variation of `script01_1.sh`. The only difference is that it should verify if a file with the same path as the second argument (output file path) already exists. If that is the case, then it should modify the file name by adding an index. As an example, let's say that the `Output` folder is empty and we call the script three time with the same list of arguments:

```
Scripts/script01_EC1.sh dataFile1.txt Output/output1.txt  
Scripts/script01_EC1.sh dataFile1.txt Output/output1.txt  
Scripts/script01_EC1.sh dataFile1.txt Output/output1.txt
```

The first time the script is executed, the file `Output/output1.txt` doesn't exist, and therefore the script calls the Version 1 executable with `Output/output1.txt` as output path.

The second time the script is executed, the script should determine that the file `Output/output1.txt` exists. It should rename this file `Output/output1 [1].txt`. Then, when it calls the Version 1 executable it should pass `Output/output1 [2].txt` as output path.

The third time the script is executed, it should find out that indexed files with the same "base name" exist, determine that the next "free" index is 3. Then, when it calls the Version 1 executable, it should pass `Output/output1 [3].txt` as output path.

4 What to submit

4.1 Organization

As stated at the beginning of this document, the folder project should be named Prog01 and will contain

- A folder named `Programs`, where all your source files should go;
- A folder named `Scripts`, where the `build.sh` (properly updated to build all the versions that you completed) will go;
- A report, as a `.pdf` file.

and will be compressed as a `zip` archive (**not** a `.rar` or other alternative archive format) that you will upload on BrightSpace.

4.2 Grading for this assignment

For this assignment, we are mostly going to go for bark over bite, pointing at problems, telling you what is going to be penalized in future assignments. So, don't just pat yourself on the back if you get a good grade, while ignoring the comments and warnings we provide, and then act all surprised when points start coming off in future assignments.

4.2.1 Execution

We are going to test your executables and script + executable pairs with different list of arguments. Your grade for this section will reflect to what extent you produce the desired output for all the test cases. So, make sure that you test your programs before you submit; don't simply replicate the examples of this handout.

Warning: Don't make **any** changes to your programs or scripts after you are done with your testing. A typical example of such last-minute changes that lead to disaster is adding a few comments and by doing so inadvertently commenting out a line of code. A program or script that has been tested should be final.

4.2.2 Code quality

For this part of the grade we look at things such as:

- Proper indentation;
- Good comments;
- Good choice of identifiers (chances are that `a`, `b`, and `c` are not good identifiers, particularly if the first two are strings and the third one an integer;

- Consistent choices of identifiers. For example, if you have a variable `int beanCount` that does what it claims to do, then it would be a bad idea to have another variable `int num_of_boxes`.
- Good implementation decision: avoid super long functions (at your current skill level, if your function occupies more than one page of your screen, it's probably too long); select proper data types; create your own when appropriate.

Your grade for this section will reflect how much you actually implemented. A superbly commented and indented “Hello World!” program won't bring you many “code quality” points.

4.2.3 No syntax error

Code with syntax errors (compiler errors for the C program) will get a grade of 0 for execution and quality. Comment out the part with syntax errors and explain in the comments what you were trying to do.

4.2.4 Report

Every assignment that you submit this semester will require a report where you explain and justify your design and implementation decisions (e.g. data structures that you chose to use to store your data in the C programs).

This assignment is fairly simple, so I don't expect a 3-page long report, but definitely more than 4-5 lines. In particular, if there are configurations under which your program may fail (crash or report invalid answers), it's always preferable for you to indicate what these configurations. At least it gives the impression that you have done some testing, identified problems, and that with some more time you may have been able to fix them.

4.2.5 Grading rubric

- C and bash code quality: 25 points
- C program execution: 45 points
 - Version 1: 20 points
 - Version 2: 25 points
 - Extra credit: 6 points
- Bash script execution: 20 points
 - Build script: 4 points
 - Run script 1: 8 points
 - Run script 2: 4 points
 - Run script 2: 4 points
 - Extra credit: 6 points
- Report: 10 points